LAPORAN TUGAS INDIVIDU

SOFTWARE TESTING - SESI 10

PROCEDURAL & OBJECT ORIENTED TEST SUITE

Studi Kasus Validasi Scope dan Pengajuan Seller ZENTA



DISUSUN OLEH

Bambang Herlambang
5803024019

Dosen Pengampu
Philipus Suryo Subandoro

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS KATOLIK WIDYA MANDALA
SURABAYA
2026

1 Tujuan dan ruang lingkup

Tugas individu ini menerapkan *basis path testing* pada satu fungsi prosedural dan *state-based testing* pada satu hierarki class. Tema diambil dari Auth marketplace kelompok ZENTA: izin client layanan dan pengajuan seller. Kode ditulis ulang secara sederhana dalam Python agar teknik perancangan test dapat ditelusuri langsung.

Batas studi kasus. Ini model lokal untuk tugas individu, bukan implementasi Auth produksi. Tidak ada login Google, JWT riil, koneksi Neon, atau perubahan role akun pada aplikasi kelompok. State DRAFT dan event withdraw merupakan asumsi model pembelajaran.

1.1 Kontrak fungsi prosedural

`validate_scopes` menerima client aktif, string scope yang diminta, dan himpunan izin. Identitas client diasumsikan telah diverifikasi oleh pemanggil. Tipe masukan adalah `bool`, `str`, dan `frozenset[str]`. Scope bersifat case-sensitive; whitespace dan duplikat dinormalisasi. Seluruh scope harus diizinkan, tidak cukup hanya sebagian. Return berupa himpunan immutable.

```python
def validate_scopes(
    client_active: bool,
    requested_scope: str,
    allowed_scopes: frozenset[str],
) -> frozenset[str]:
    """Terima hanya client aktif dengan scope tidak kosong dan seluruhnya diizinkan.

    Precondition: client_active adalah bool, requested_scope adalah str, dan
    allowed_scopes adalah frozenset[str]. Identitas client sudah diverifikasi
    oleh pemanggil. Scope bersifat case-sensitive; whitespace dan duplikat
    dinormalisasi. Fungsi tidak mengubah konfigurasi izin milik pemanggil.
    """
    if not client_active:  # D1
        raise ScopeValidationError("CLIENT_INACTIVE")

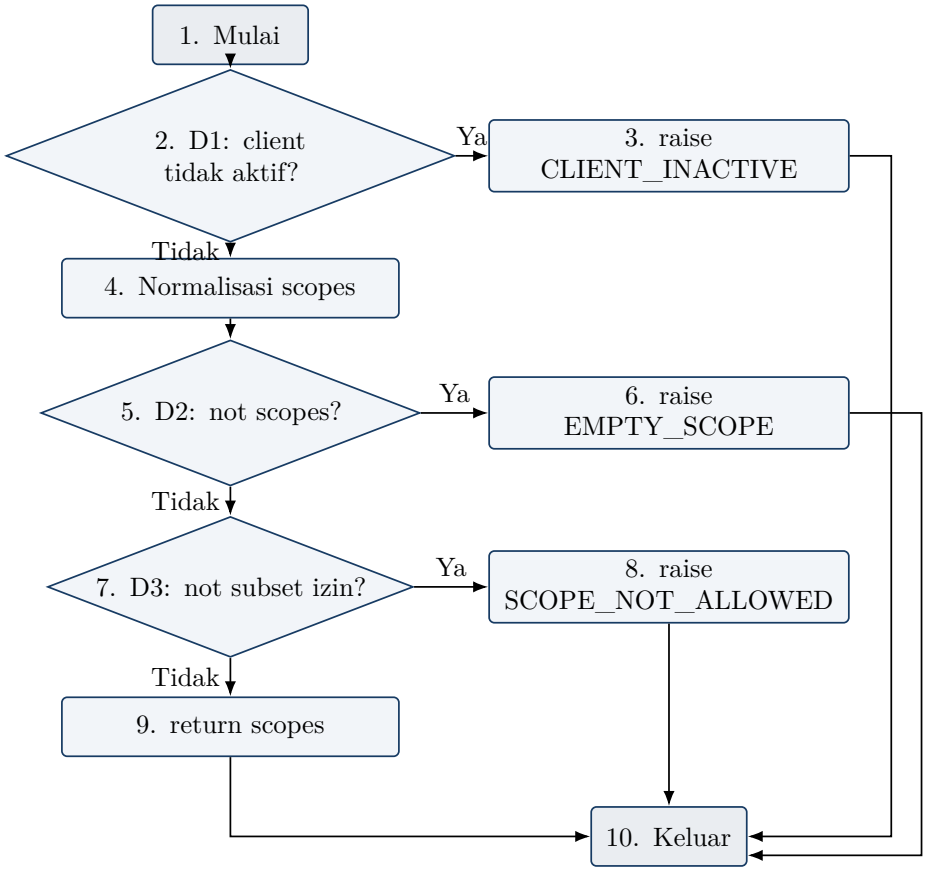
    scopes = frozenset(requested_scope.split())
    if not scopes:  # D2
        raise ScopeValidationError("EMPTY_SCOPE")

    if not scopes.issubset(allowed_scopes):  # D3
        raise ScopeValidationError("SCOPE_NOT_ALLOWED")

    return scopes
```

Tiga keputusan diberi tanda D1, D2, dan D3. Urutan error adalah `CLIENT_INACTIVE`, `EMPTY_SCOPE`, lalu `SCOPE_NOT_ALLOWED`. Tidak ada secret yang diperlukan untuk test.

2 Control flow graph dan cyclomatic complexity



N = 10, E = 12, P = 1. Node 10 adalah exit virtual bersama bagi return dan exception; operasi bawaan Python diperlakukan sebagai blok tunggal.

$$V(G) = E - N + 2P = 12 - 10 + 2 = \boxed{4} \qquad V(G) = D + 1 = 3 + 1 = \boxed{4}$$

Keempat jalur pada halaman berikut membentuk basis: setiap jalur baru memperkenalkan edge yang belum dilalui. Minimum basis path adalah 4 test untuk fungsi ini, dengan mempertimbangkan jalur yang dapat dieksekusi.

3 Jalur independen dan test prosedural

Jalur	Urutan node	D1/D2/D3	Hasil yang diharapkan
P1	1-2-3-10	T / – / –	CLIENT_INACTIVE
P2	1-2-4-5-6-10	F / T / –	EMPTY_SCOPE
P3	1-2-4-5-7-8-10	F / F / T	SCOPE_NOT_ALLOWED
P4	1-2-4-5-7-9-10	F / F / F	Himpunan scope sah

T/F merujuk ekspresi negatif pada kode. P2 menambahkan edge 2–4, P3 menambahkan 5–7, dan P4 menambahkan 7–9. Kombinasi T/T/T tidak feasible karena error menyebabkan fungsi langsung keluar.

3.1 Empat test basis path

Cuplikan berikut diambil langsung dari `tests/test_scopes.py`. Fixture `order_scopes` berisi `read:catalog`, `reserve:stock`, `commit:stock`, dan `release:stock`.

```
def test_p1_inactive_client_is_rejected(order_scopes):
    with pytest.raises(ScopeValidationError, match="^CLIENT_INACTIVE$"):
        validate_scopes(False, "read:catalog", order_scopes)

def test_p2_empty_scope_is_rejected(order_scopes):
    with pytest.raises(ScopeValidationError, match="^EMPTY_SCOPE$"):
        validate_scopes(True, "", order_scopes)

def test_p3_disallowed_scope_is_rejected(order_scopes):
    with pytest.raises(ScopeValidationError, match="^SCOPE_NOT_ALLOWED$"):
        validate_scopes(True, "admin:users", order_scopes)

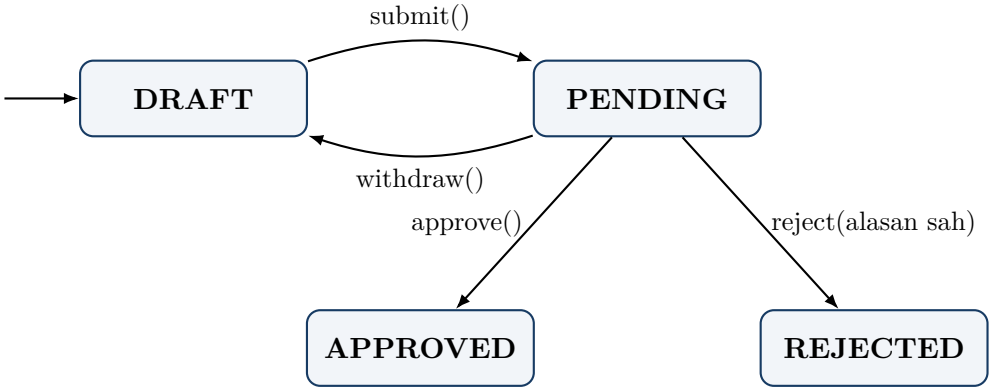
def test_p4_allowed_scopes_are_returned(order_scopes):
    result = validate_scopes(True, "reserve:stock commit:stock release:stock", order_scopes)
    assert result == frozenset({"reserve:stock", "commit:stock", "release:stock"})
```

3.2 Data flow dan test tambahan

Variabel `scopes` didefinisikan pada node 4, dipakai pada predicate node 5 dan 7, lalu dikembalikan pada node 9. P2, P3, dan P4 melintasi penggunaan tersebut. Test tambahan memeriksa whitespace, duplikat, nama scope utuh, case sensitivity, campuran izin sah/tidak sah, allowlist kosong, prioritas error, dan allowlist yang tidak berubah. Total bagian ini **14 kasus**. Tidak ada loop eksplisit yang perlu diklaim teruji.

4 State machine class pengajuan seller

Class `SellerApplication` memiliki empat state. Nama toko wajib non-kosong. Status dan alasan penolakan dapat dibaca melalui property publik; test tidak mengakses atribut privat. APPROVED dan REJECTED merupakan state terminal untuk objek pengajuan yang sama.



4.1 Tabel transisi lengkap

State awal	submit	approve	reject	withdraw
DRAFT	PENDING	Error	Error	Error
PENDING	Error	APPROVED	REJECTED	DRAFT
APPROVED	Error	Error	Error	Error
REJECTED	Error	Error	Error	Error

Error pada tabel berarti `InvalidTransitionError`, dengan status, nama toko, dan alasan penolakan tetap seperti sebelum event. Untuk kolom reject, tabel mengasumsikan alasan non-kosong.

- Terdapat 4 pasangan valid dan 12 pasangan tidak valid per implementasi.
- Reject dari PENDING dengan alasan kosong/whitespace menimbulkan `ValueError`; status tetap PENDING dan alasan tetap None.
- DRAFT, withdraw, dan kategori GENERAL adalah asumsi khusus tugas ini. Persetujuan tidak membuat toko atau mengubah hak akses akun riil.

5 Pewarisan dan polimorfisme

Dua subclass mengkhususkan `category()` dan mewarisi seluruh method transisi. Class induk mengembalikan GENERAL, sedangkan subclass mengembalikan FOOD dan CLOTHING. Kontrak bersama tetap kategori string non-kosong serta aturan state dan exception yang sama.

```
class FoodSellerApplication(SellerApplication):
    """Mewarisi seluruh aturan transisi, mengkhususkan label kategori."""

    def category(self) -> str:
        return "FOOD"

class ClothingSellerApplication(SellerApplication):
    """Mewarisi seluruh aturan transisi, mengkhususkan label kategori."""

    def category(self) -> str:
        return "CLOTHING"
```

Fixture `application_class` berparameter class induk dan kedua subclass. Karena itu setiap test matriks diuji tiga kali: **16 x 3 = 48 kasus**. Method warisan tidak diasumsikan benar hanya karena class induk lulus. Subclass tidak menambah prasyarat ataupun mengubah hasil transisi induk.

```
@pytest.fixture(params=[SellerApplication,
                        FoodSellerApplication,
                        ClothingSellerApplication])
def application_class(request):
    return request.param

applications: list[SellerApplication] = [
    FoodSellerApplication("Dapur Demo"),
    ClothingSellerApplication("Sandang Demo"),
]
assert [a.category() for a in applications] == ["FOOD", "CLOTHING"]
for application in applications:
    application.submit()
    application.approve()
assert [a.status for a in applications] == [S.APPROVED, S.APPROVED]
```

Cuplikan kedua merangkum fixture dan test koleksi polimorfik; kode lengkap berada pada `tests/test_seller.py`. Pemanggil memakai interface yang sama, tetapi label kategori ditentukan oleh implementasi konkret.

6 Perancangan test state-based

6.1 Oracle dan isolasi objek

Tabel berisi 16 pasangan state/event ditulis eksplisit pada `TRANSITIONS`; expected result tidak dihitung dari implementasi. Helper `make_application` mencapai state awal dengan method publik, bukan menulis atribut privat. Setiap test mendapat objek baru.

```
def test_entire_transition_contract(application_class, initial, event, expected):
    application = make_application(application_class, initial)
    before = snapshot(application)
    action = getattr(application, event)
    args = ("  Dokumen tidak sesuai  ",) if event == "reject" else ()

    if expected is None:
        with pytest.raises(InvalidTransitionError):
            action(*args)
        assert snapshot(application) == before  # kegagalan tidak merusak state
    else:
        assert action(*args) is None
        assert application.status == expected
        assert application.shop_name == "Kedai Demo"
        if expected == S.REJECTED:
            assert application.rejection_reason == "Dokumen tidak sesuai"
        else:
            assert application.rejection_reason is None
```

Fungsi tersebut dijalankan oleh `parametrize` dengan 16 baris tabel dan fixture 3 class. Snapshot hanya membaca kontrak publik. Untuk operasi gagal, test memeriksa exception sekaligus seluruh snapshot agar penolakan tidak meninggalkan perubahan sebagian.

6.2 Kasus di luar matriks

- Nama toko dinormalisasi dan input kosong ditolak saat konstruksi.
- Alasan penolakan harus bermakna; spasi saja tidak mengubah PENDING.
- Urutan submit–withdraw–submit–approve berakhir pada APPROVED.
- Dua instance memiliki state independen; status publik tidak bisa diisi langsung untuk melewati transisi.
- Override kategori dan pemanggilan polimorfik diverifikasi terpisah.

Total pengujian OO adalah **82 kasus**, termasuk 48 kasus matriks. Test scope dan OO digabung menjadi 96 kasus setelah parametrization.

7 Hasil eksekusi dan analisis coverage

Hasil aktual lokal: 96 test lulus, 0 gagal. Eksekusi: `2026-11-02T15:24:05+07:00` (UTC+07:00). Lingkungan: Windows, Python 3.10.11, PyTest 8.4.2, pytest-cov 7.0.0, coverage.py 7.10.7. Angka ringkasan ini dihasilkan dari JSON hasil eksekusi; source/test dicek SHA-256 sebelum PDF dibangun.

7.1 Ringkasan per modul

Modul	Stmt	Miss	Branch	MissBr	Branch %
`__init__.py`	0	0	0	0	–
`scopes.py`	10	0	6	0	100%
`seller.py`	52	0	12	0	100%
Total	62	0	18	0	100%

Stmt = statement executable; Branch = kemungkinan tujuan cabang. Tanda – berarti modul tidak memiliki cabang, sehingga persentase branch tidak memiliki denominator. Tidak ada statement atau branch yang sengaja dikecualikan dari pengukuran.

$$\text{Statement coverage} = \frac{62}{62} \times 100\% = 100\%$$

$$\text{Branch coverage} = \frac{18}{18} \times 100\% = 100\%$$

```
Name                          Stmts   Miss Branch BrPart    Cover   Missing
-------------------------------------------------------------------------
src\zenta_auth\__init__.py        0      0      0      0  100.00%
src\zenta_auth\scopes.py         10      0      6      0  100.00%
src\zenta_auth\seller.py         52      0     12      0  100.00%
-------------------------------------------------------------------------
TOTAL                            62      0     18      0  100.00%
```

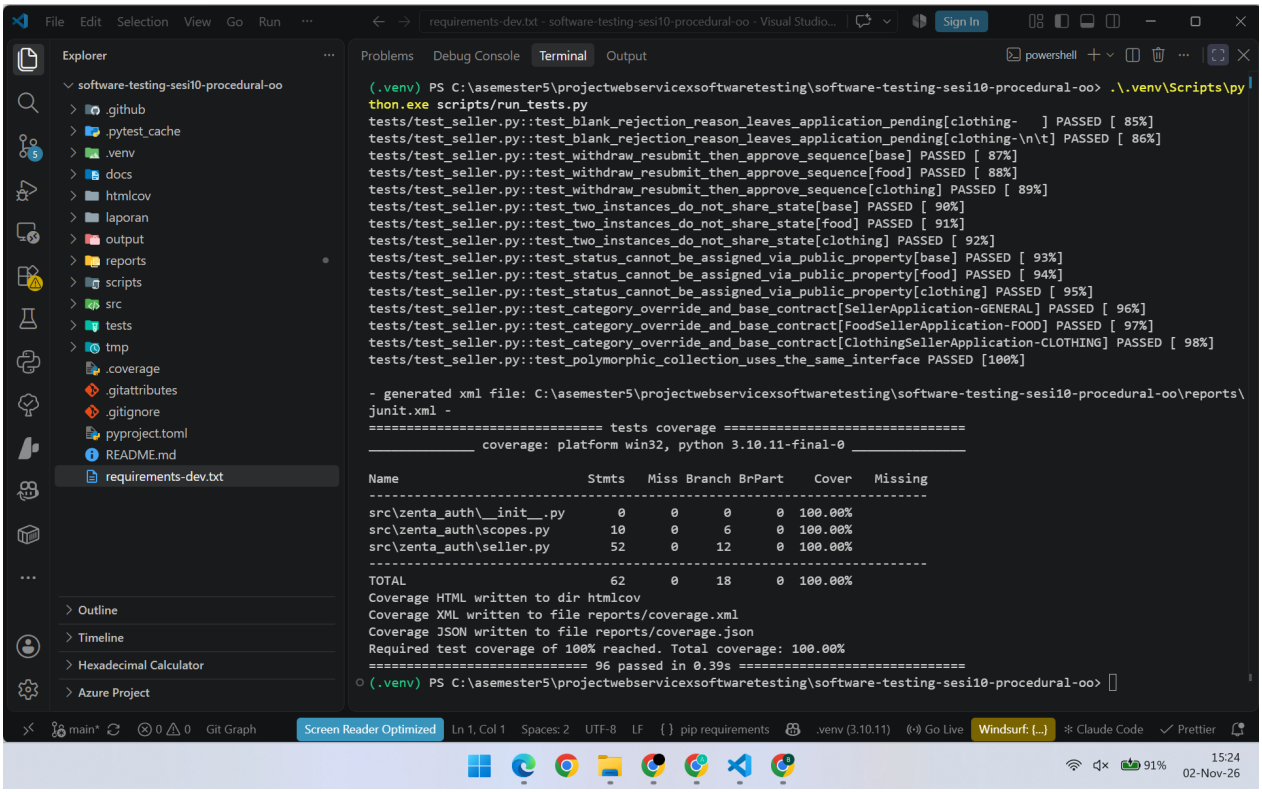
7.2 Makna dan batas hasil

Fungsi scope menutup enam tujuan cabang dari tiga keputusan. Modul seller menutup dua tujuan untuk setiap guard: nama toko, submit, approve, reject-state, reject-alasan, dan withdraw. Kolom `Cover` terminal menggabungkan peluang statement dan branch; persentase branch tersendiri dihitung dari `covered_branches/num_branches` pada JSON.

Pengukuran hanya mencakup `src/zenta_auth`; test, script, dan library tidak masuk denominator. Coverage 100% berarti seluruh statement/cabang model terlintasi, bukan bukti seluruh input, semua urutan tak terbatas, atau keamanan Auth produksi benar. Tidak ada test integrasi Google/Neon. Laporan HTML menyediakan detail per baris, XML/JUnit untuk alat CI, dan metadata menyimpan versi serta hash source/test.

7.3 Bukti eksekusi PyTest di terminal

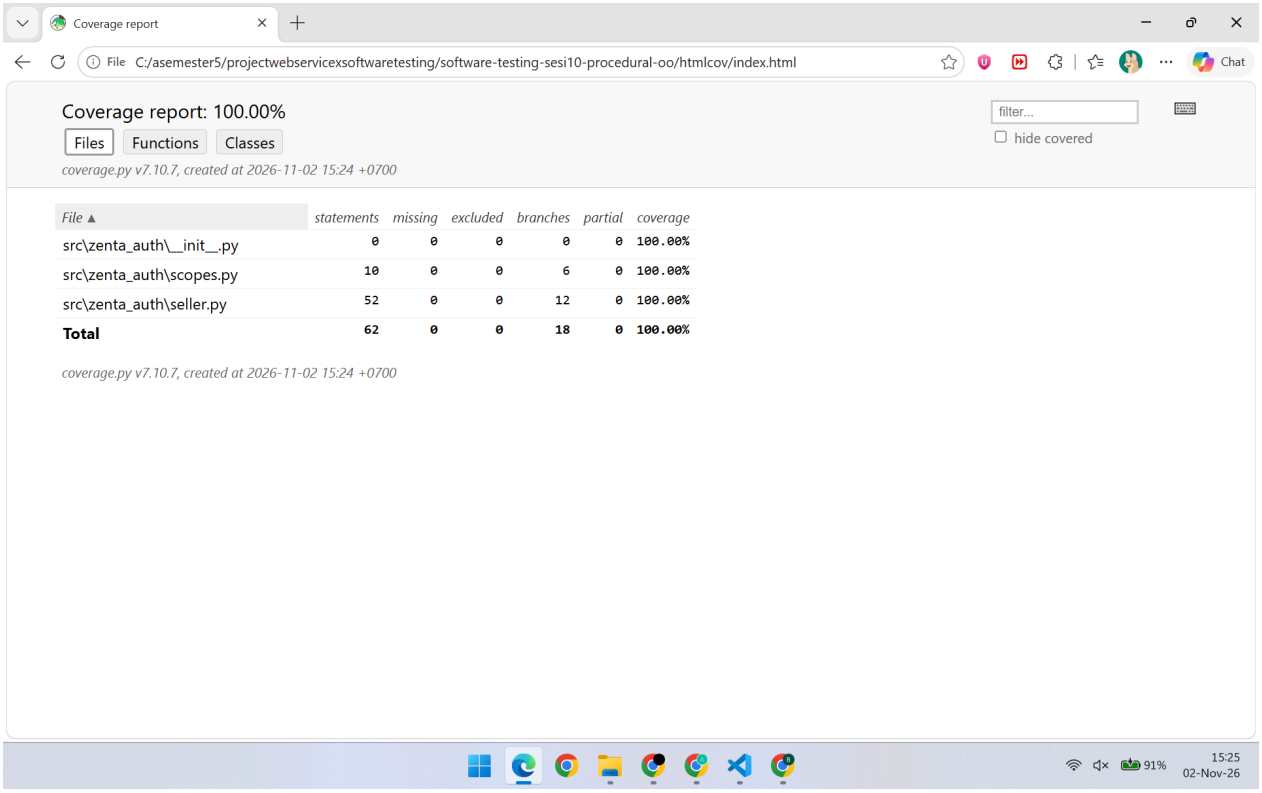
Gambar 1 memperlihatkan hasil eksekusi oleh mahasiswa: 96 test berstatus lulus dalam 0,39 detik. Ringkasan coverage mencatat 62 statement dan 18 tujuan cabang, tanpa statement hilang maupun cabang parsial. Baris keluaran juga menunjukkan pembuatan laporan HTML, XML, JSON, dan JUnit. Hasil ini mendukung ringkasan pengujian lokal pada bagian sebelumnya.



Gambar 1: Hasil pengujian dan coverage pada terminal, ditampilkan utuh.

7.4 Bukti coverage per modul pada tab Files

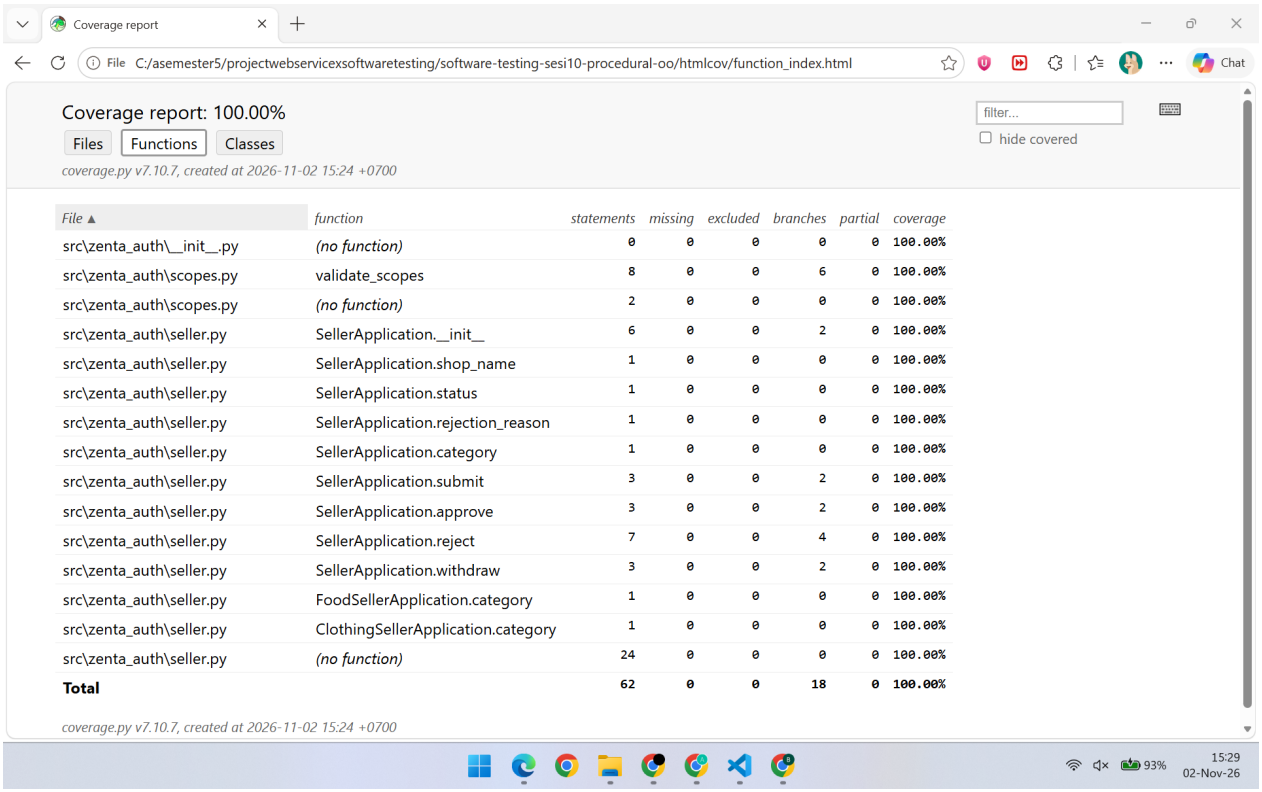
Gambar 2 menampilkan ringkasan HTML dari eksekusi yang sama. Modul `scopes.py` memiliki 10 statement dan 6 tujuan cabang; `seller.py` memiliki 52 statement dan 12 tujuan cabang. Semua tercakup, sehingga totalnya 62 statement dan 18 tujuan cabang dengan coverage 100%. Kolom *missing*, *excluded*, dan *partial* bernilai nol; file `__init__.py` tidak memiliki statement yang diukur.



Gambar 2: Ringkasan coverage per file pada tab Files, ditampilkan utuh.

7.5 Bukti coverage fungsi dan method pada tab Functions

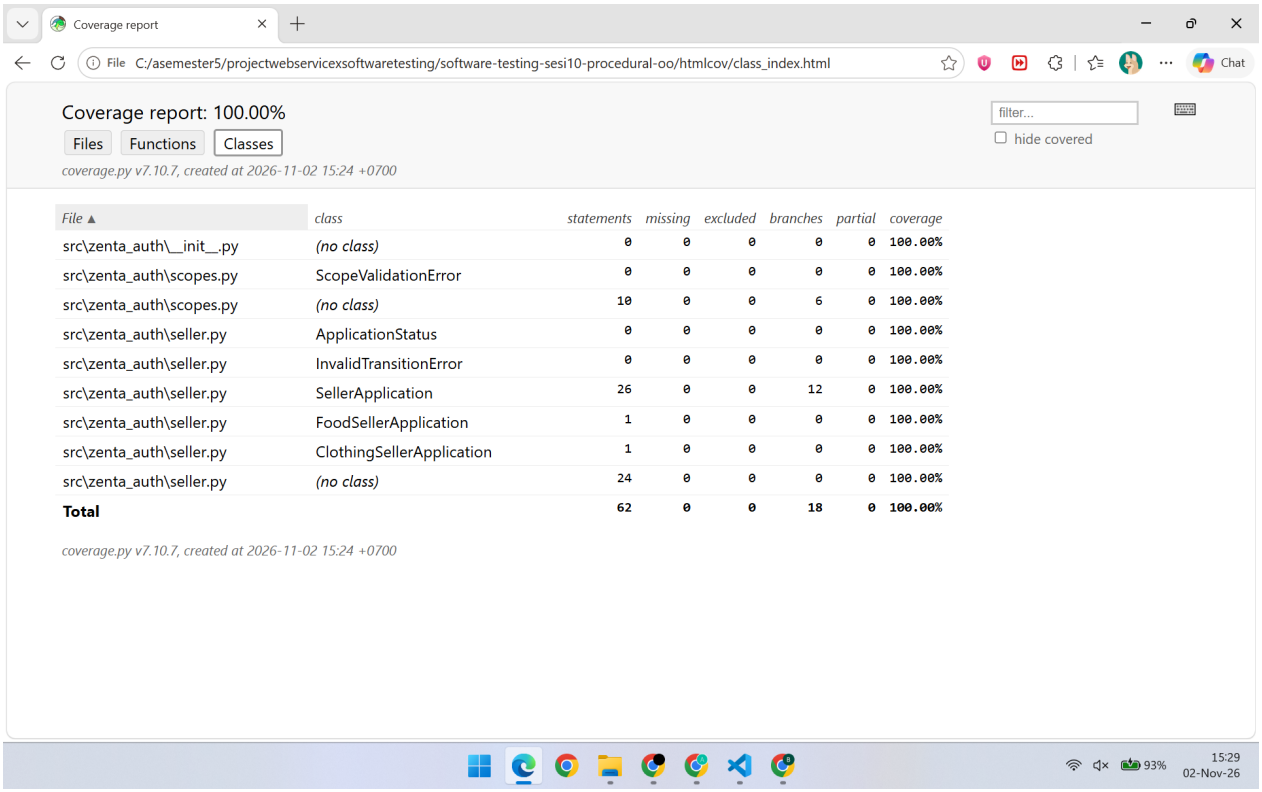
Gambar 3 merinci bagian kode yang dijalankan di setiap fungsi. `validate_scopes` mencakup 8 statement dan 6 tujuan cabang; dua statement lain pada modul scope berada di luar fungsi. Method `reject` memiliki 4 tujuan cabang karena memeriksa state dan alasan penolakan, sedangkan `submit`, `approve`, dan `withdraw` masing-masing memiliki 2. Seluruhnya tercakup; ketepatan hasil tetap diperiksa melalui assertion test.



Gambar 3: Rincian coverage fungsi dan method pada tab Functions, ditampilkan utuh.

7.6 Bukti coverage class pada tab Classes

Gambar 4 menunjukkan `SellerApplication` mencakup 26 statement dan 12 tujuan cabang. Kedua subclass masing-masing mencakup satu statement pada override kategori. Baris *(no class)* memuat kode di luar method class, sehingga angka class tidak sama dengan total modul. Class tanpa statement terukur tampil 100% menurut alat; bukti perilaku pewarisan dan polimorfisme berasal dari test kontrak serta urutan transisi, bukan persentase coverage saja.



Gambar 4: Rincian coverage class pada tab Classes, ditampilkan utuh.

8 Reproduksi dan pemetaan tugas

Jalankan dari root repo individu. Contoh PowerShell berikut tidak memerlukan aktivasi environment ataupun perubahan ExecutionPolicy Windows.

```
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements-dev.txt
.\.venv\Scripts\python.exe scripts/run_tests.py
Start-Process .\htmlcov\index.html
```

Perintah pengukuran utamanya adalah:

```
python -m pytest --cov=zenta_auth --cov-branch --cov-report=term-missing
```

Untuk membangun ulang PDF setelah pengujian, pasang Tectonic, lalu jalankan `python scripts/build_report.py`. Jika binary tidak di PATH, tambahkan opsi `-tectonic` diikuti lokasi binary. PDF disimpan pada `output/pdf/LaporanSesi10BambangHerlambang.pdf`. README memuat perintah lengkap. Template GitHub Actions tersedia di `docs/ci-tests.yml` untuk menjalankan test dan mengunggah artefak coverage. CI belum diaktifkan saat laporan ini dibuat; angka di atas merupakan hasil eksekusi lokal. CI bukan syarat tugas ini.

Butir penilaian	Bukti yang disertakan
CFG, V(G), jalur independen (30%)	3 keputusan, N=10, E=12, V(G)=4, empat basis path dan test P1–P4.
State valid/tidak valid (30%)	Diagram 4 state, matriks 16 pasangan, 48 eksekusi matriks lintas 3 class.
Inheritance/polimorfisme (15%)	Dua override kategori, kontrak method warisan diuji ulang, koleksi dengan interface induk.
Kerapian dan coverage (25%)	Fixture, parametrization, exception oracle, 96 test lulus, branch 18/18, bukti JSON/XML/HTML.

9 Referensi

[1] Philipus Suryo Subandoro. Materi Software Testing Sesi 10: *Procedural Testing & Object Oriented Testing*. Teks tugas yang diberikan mahasiswa; digunakan sebagai sumber rubrik dan tujuan pembelajaran.

[2] McCabe, T. J. (1976). A Complexity Measure. *IEEE Transactions on Software Engineering*, SE-2(4), 308–320. DOI: 10.1109/TSE.1976.233837.

[3] PyTest. *How to parametrize fixtures and test functions*. https://docs.pytest.org/en/stable/how-to/parametrize.html.

[4] pytest-cov. *Configuration*, opsi `-cov-branch`. https://pytest-cov.readthedocs.io/en/latest/config.html.

[5] coverage.py. *Branch coverage measurement*. https://coverage.readthedocs.io/en/latest/branch.html.